

ASG Airlines: End-to-End Data Engineering Case Study

Documentation & Technical Report

Harsh Prasad

September 12, 2026

Contents

1	Problem Overview	3
2	Dataset Structure	3
2.1	Final Cleaned Schema	3
3	Architecture, Data Flow & Data Model	4
3.1	Architecture Diagram	4
3.2	Data Flow Diagram	4
3.3	Data Model	5
4	Assumptions	6
5	Data Cleaning Logic & Strategy	6
5.1	Text Standardization	6
5.2	Missing Value Handling — Airline Column	6
5.3	flight_id Validation	6
5.4	Duplicate Removal	6
5.5	Timestamp Validation	7
6	Transformation Logic	7
6.1	Corrupted Timestamp Detection	7
6.2	Overnight Flight Detection	7
6.3	Duration Calculation	7
6.4	Anomaly Flagging	7
7	PII & Privacy	8
7.1	Access Control Recommendation	8
8	Key Performance Indicators	8
9	Power BI Dashboard	9
9.1	Overview	9
9.2	Duration Analysis	9
9.3	Route Performance	10
9.4	Airline Trends	10
9.5	Delay / Anomaly Insights	11

1 Problem Overview

ASG Airlines operates flights across multiple Indian cities and collects operational flight data from its booking platforms, scheduling systems, and airport logs. The raw dataset supplied for this case study contains intentional data quality issues, including corrupted or inconsistent timestamps, missing airline values, duplicate records, and overnight (cross-day) flights that were not correctly represented.

The objective of this project was to design and implement an end-to-end data pipeline that ingests the raw flight data, cleans and standardizes it, correctly resolves overnight flight scenarios, computes key operational KPIs, and presents the results through an interactive Power BI dashboard.

This document describes the dataset structure, the architecture and data flow of the pipeline, the assumptions made during cleaning, the transformation logic applied (including the overnight-flight handling approach), the privacy considerations, and a walkthrough of the final dashboard.

2 Dataset Structure

The raw dataset was supplied as an Excel file (`UseCase - Airlines.xlsx`) containing **1,020 records** across **7 columns**.

Column	Type	Description
flight_id	string	Flight identifier, format: 2-character alphanumeric airline code + 3 digits (e.g. AI155, 6F026, SJ010, UK094).
airline	string	Operating airline name. Contained missing values and the literal string "UNKNOWN" in place of a proper null.
source	string	Departure airport code. Six distinct codes present: CCU, BOM, MAA, DEL, BLR, HYD.
destination	string	Arrival airport code. Same six codes as source.
departure_time	datetime	Full departure timestamp (date + time).
arrival_time	datetime	Full arrival timestamp (date + time).
duration	mixed	Raw duration field. Inconsistently typed by Excel (a mix of HH:MM:SS strings and Excel serial datetime artifacts), and therefore not used directly.

Table 1: Original (raw) dataset schema

Four airlines are represented in the data: **SpiceJet, Air India, Vistara, and IndiGo**. Each airline's `flight_id` values share a consistent 2-character prefix (SJ, AI, UK, and 6F respectively), which proved useful during cleaning (see Section 4).

2.1 Final Cleaned Schema

After the cleaning and transformation pipeline (Section 5), the dataset was reduced to **1,004 records** and enriched with the following derived columns:

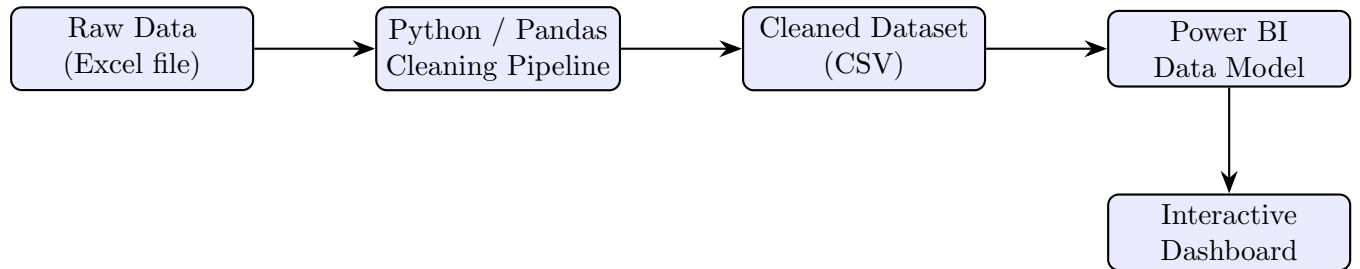
Column	Type	Description
route	string	Concatenation of source and destination (e.g. BOM-CCU), used for route-level aggregation.
dep_date	date	Calendar date extracted from departure_time .
dep_hour	integer	Hour-of-day extracted from departure_time , used to study time-of-day patterns.
overnight	boolean	True if the flight's arrival date differs from its departure date.
duration_min	float	Flight duration in minutes, recalculated directly from timestamps rather than trusting the raw duration column.
anomaly	boolean	True if duration_min ≤ 0 or > 360 minutes.

Table 2: Final cleaned dataset schema (`flights_cleaned.csv`)

3 Architecture, Data Flow & Data Model

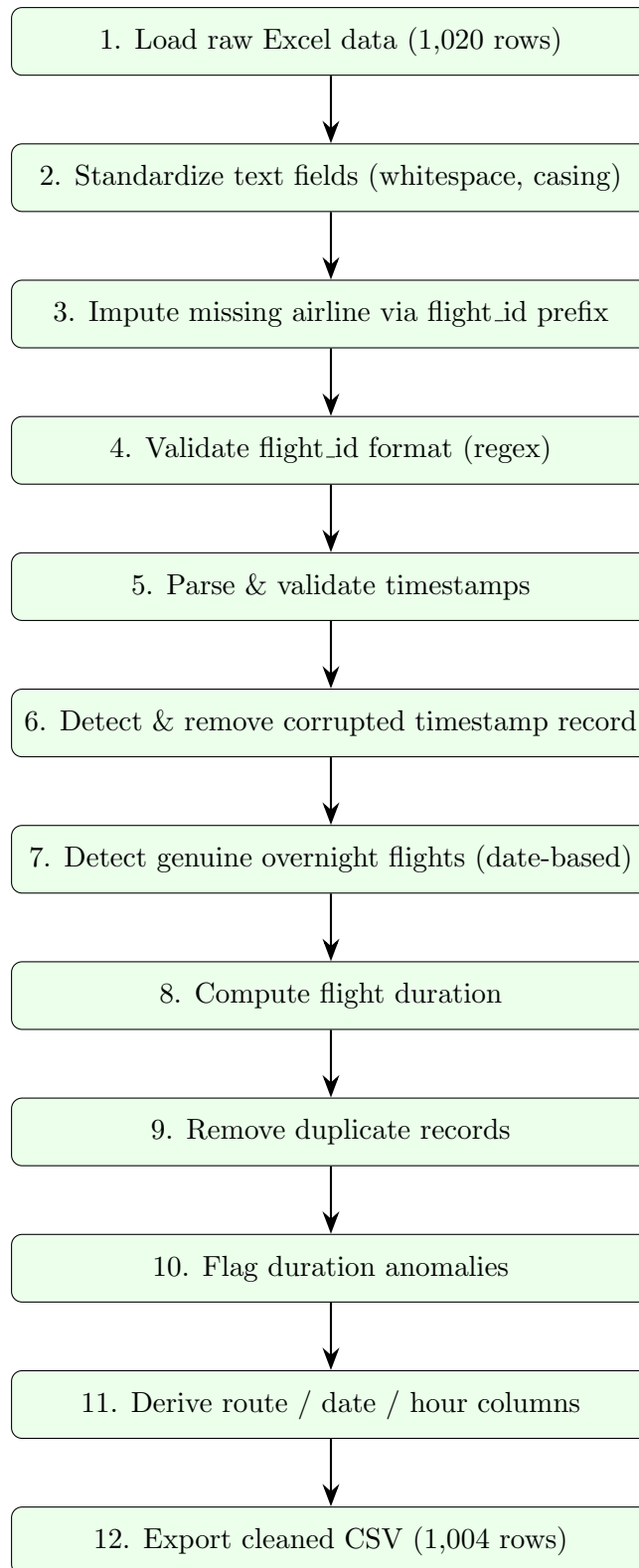
3.1 Architecture Diagram

The pipeline was implemented locally using Python (pandas) rather than Azure cloud services, in line with the case study's stated alternative option. The architecture is intentionally simple and reproducible:



3.2 Data Flow Diagram

The transformation flow within the notebook follows this sequence:



3.3 Data Model

The final Power BI data model is a simple, single-fact-table design:

- **flights_cleaned** — the fact table, one row per flight, containing all dimensions (airline, route, source, destination) and measures (duration, anomaly, overnight flags) needed for reporting.

- **DateTable** — a standalone calendar dimension table (generated via `CALENDAR()` over the departure date range), used to support date-based slicing independently of the fact table.

No formal relationship join was required beyond what Power BI's auto-detected date hierarchy provides, since the dataset's short date range (4 days) did not require complex time-intelligence calculations.

4 Assumptions

1. Each `flight_id` prefix uniquely and reliably identifies an airline (`AI` → Air India, `SJ` → SpiceJet, `UK` → Vistara, `6F` → IndiGo). This assumption was validated by checking that every prefix in the dataset mapped cleanly to exactly one of the four known airlines, with zero unmapped cases.
2. A flight duration of **0 minutes or less** is invalid, and a duration **greater than 360 minutes (6 hours)** is anomalous, given that all routes in this dataset are domestic and geographically close (Mumbai, Delhi, Bangalore, Chennai, Hyderabad, Kolkata).
3. The raw `duration` column was not trusted as a source of truth, since it exhibited inconsistent typing when read from Excel (a mix of `datetime.time` objects and full `datetime` objects). Duration was instead recalculated directly from the `departure_time` and `arrival_time` timestamps.
4. A record where `arrival_time` falls on a calendar date *earlier* than `departure_time` is not a valid overnight flight — it is treated as a corrupted or invalid record and removed.

5 Data Cleaning Logic & Strategy

5.1 Text Standardization

All text fields (`flight_id`, `airline`, `source`, `destination`) were stripped of leading/trailing whitespace, and `flight_id`, `source`, and `destination` were upper-cased to eliminate casing inconsistencies.

5.2 Missing Value Handling — Airline Column

The `airline` column contained 72 missing/invalid entries, disguised in three separate forms: the literal string "UNKNOWN", the string "nan", and empty strings. These were first unified into a single proper null representation. The missing values were then recovered using the `flight_id` prefix as a lookup key against a manually defined mapping (`AI`, `SJ`, `UK`, `6F`). This prefix-based imputation resolved **100% of the missing airline values** — a stronger outcome than simply dropping the affected rows.

5.3 flight_id Validation

A regular expression (`^[A-Z0-9]{2}\d{3}$`) was applied to validate the expected `flight_id` format. All 1,020 records passed this check; no corrupted identifiers were found in this particular dataset, though the validation step remains in the pipeline as a safeguard for future data batches.

5.4 Duplicate Removal

15 exact duplicate rows were identified via `.duplicated()` and removed based on the combination of `flight_id`, `departure_time`, and `arrival_time`.

5.5 Timestamp Validation

Both `departure_time` and `arrival_time` were coerced to proper datetime objects using `pd.to_datetime(..., errors= 'coerce')`. Any row where either value could not be parsed would be dropped entirely, since a flight without a valid timestamp cannot be used for duration or KPI calculations. No such rows were present in this dataset.

6 Transformation Logic

6.1 Corrupted Timestamp Detection

Before applying overnight-flight logic, the pipeline checks for records where `arrival_time` is chronologically *before* `departure_time` as a full timestamp comparison. Only one such record was found (`flight_id` SJ192), with an arrival date one full day *earlier* than its departure date — a physically impossible sequence. This is a data corruption case, not an overnight flight, and no date-adjustment can correct it. The record was removed from the dataset prior to further processing.

6.2 Overnight Flight Detection

An earlier implementation attempted to detect overnight flights by comparing full departure and arrival timestamps directly (`arrival_time < departure_time`). This approach returned only a single flagged row — the corrupted SJ192 record described above — rather than the expected population of genuine overnight flights.

Investigation of the raw data revealed that this dataset already embeds the *correct* calendar date on `arrival_time` for flights that cross midnight (e.g. departure 2026-04-20 23:38, arrival 2026-04-21 02:32). Comparing full timestamps therefore correctly evaluates such cases as arrival being *later* than departure, and does not flag them — which was the source of the discrepancy.

The corrected approach compares only the **calendar date** component of each timestamp:

```
overnight = arrival_time.dt.date != departure_time.dt.date
```

This correctly identified **124 overnight flights** prior to deduplication (**122** in the final deduplicated dataset), matching the expected pattern of late-night departures landing after midnight.

6.3 Duration Calculation

Since the corrupted record was removed and the dataset's arrival timestamps already carry the correct calendar date for overnight flights, duration is computed via a direct, vectorized subtraction:

```
duration_min = (arrival_time - departure_time).dt.total_seconds() / 60
```

No manual "+1 day" adjustment is required, since the date component is already correct in the source data once the single corrupted record is excluded.

6.4 Anomaly Flagging

Following duration calculation, each flight is flagged as anomalous if:

```
duration_min <= 0 OR duration_min > 360
```

After cleaning, **zero records** in the final dataset meet this condition — the sole anomalous record (SJ192) had already been identified and removed as a corrupted timestamp earlier in the

pipeline. This is reported as a genuine finding: the cleaning process resolved the dataset’s only real anomaly upstream of the anomaly-flagging step itself.

7 PII & Privacy

The dataset operates strictly at the **flight-operations level** — it contains no passenger names, contact details, booking references, payment information, or any other personally identifiable information. The `flight_id` field is an operational identifier assigned by the airline, not personal data.

As a precaution, the pipeline explicitly checks all column names for PII-indicative keywords (`name`, `passenger`, `email`, `phone`, `pnr`, `address`, `card`) before finalizing the schema. No such columns were found, and no masking, hashing, or encryption was therefore required for this dataset.

7.1 Access Control Recommendation

Although this dataset contains no PII, if passenger-level data were to be incorporated into this pipeline in the future (e.g. booking or passenger manifests), the following access-control measures would be recommended:

- Role-based access control (RBAC) on the Power BI workspace, restricting passenger-level reports to authorized operations staff only.
- Row-level security (RLS) rules scoped by department or route, so that only relevant stakeholders can view granular data.
- Masking or hashing of any passenger identifiers at the ingestion stage, before the data reaches the reporting layer.

8 Key Performance Indicators

The following KPIs were computed from the final cleaned dataset (1,004 records):

KPI	Value
Total Flights	1,004
Average Flight Duration	164.49 minutes
Overnight Flights	122
Anomalous Flights	0
Total Unique Routes	30
Flights by Airline	IndiGo: 273, Air India: 255, SpiceJet: 246, Vistara: 230
Busiest Route	BOM–CCU (90 flights)
Busiest Source Airport	BOM
Busiest Destination Airport	DEL

Table 3: Summary KPIs

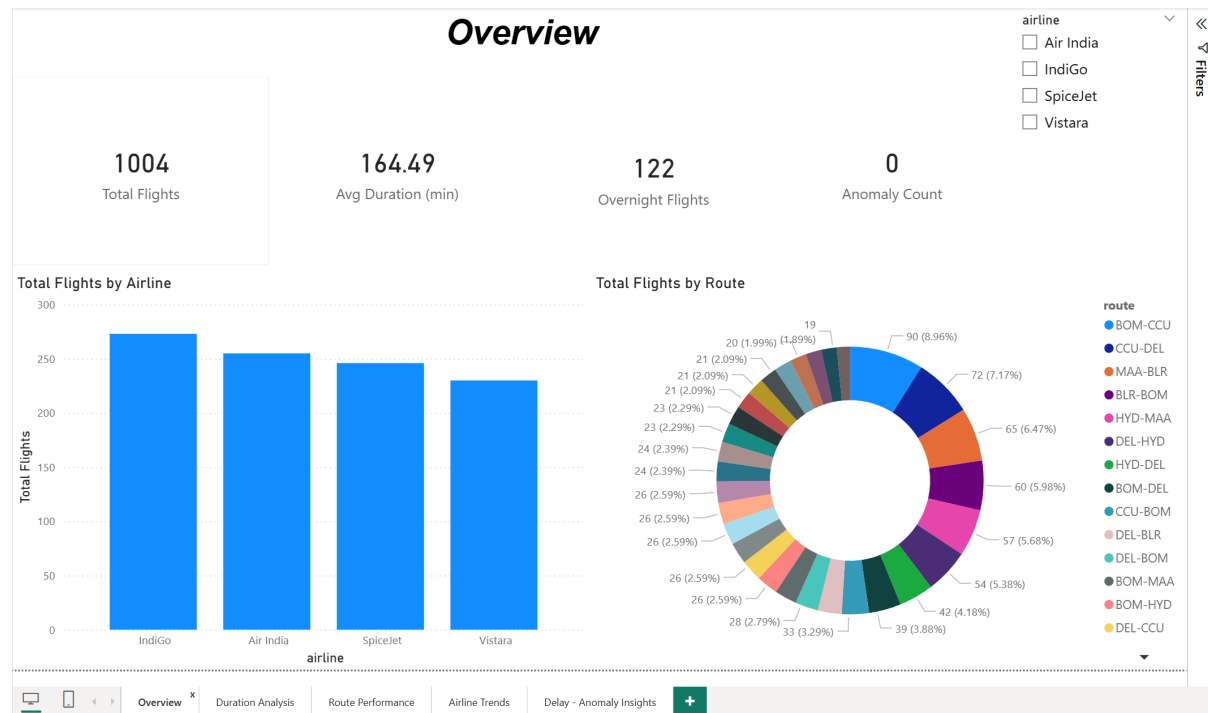
Additional KPIs computed beyond the baseline requirement include: average duration by route, overnight-flight breakdown by airline and by route, and flight-count trends by day and airline — all incorporated into the dashboard described in the next section.

9 Power BI Dashboard

The dashboard is organized into five pages, each addressing a distinct operational lens on the cleaned dataset, with synchronized `airline` filters and interactive slicers throughout.

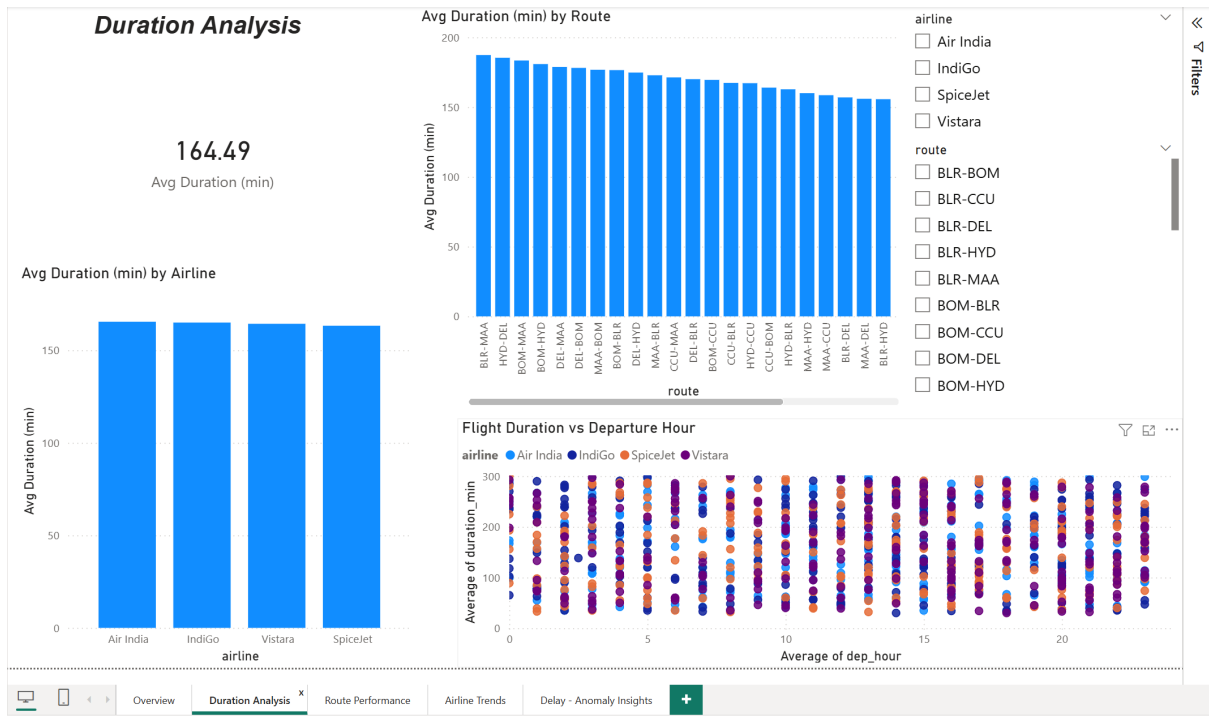
9.1 Overview

Provides headline KPI cards (Total Flights, Average Duration, Overnight Flights, Anomaly Count) alongside a flight-count breakdown by airline and a route-share donut chart, giving an at-a-glance operational summary.



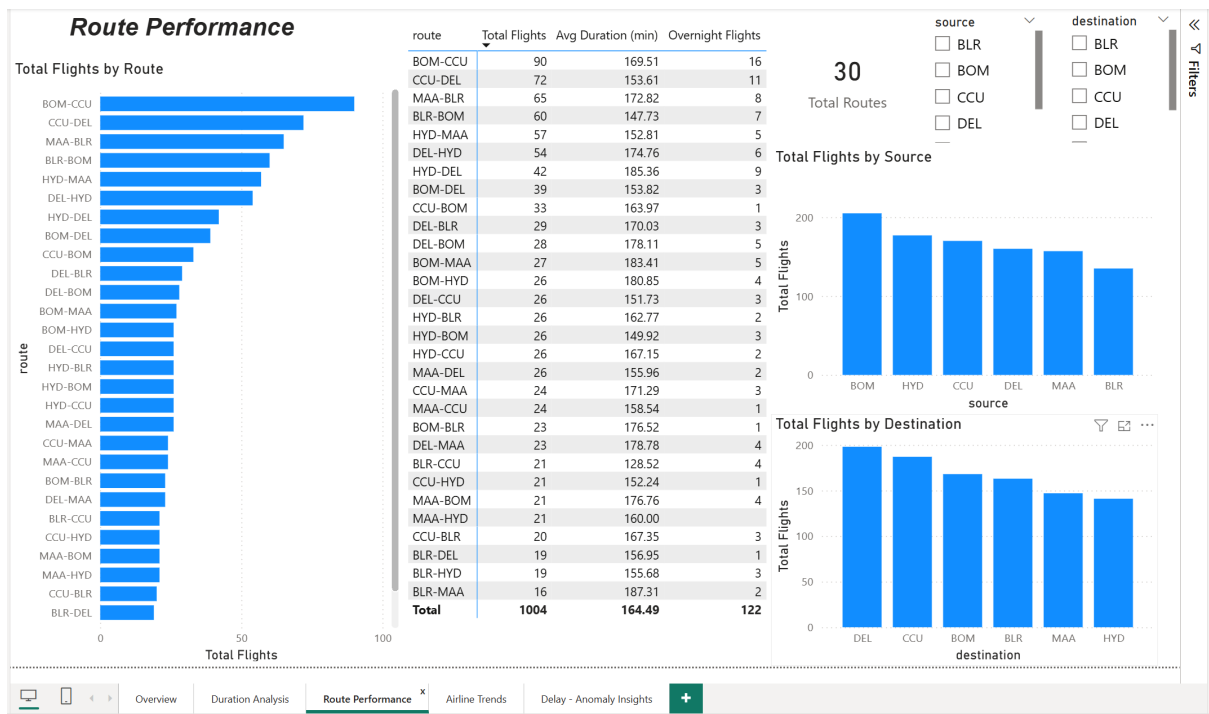
9.2 Duration Analysis

Breaks down average flight duration by airline and by route (sorted descending to surface the longest routes), and includes a disaggregated scatter chart of duration against departure hour, colored by airline, to examine whether time-of-day correlates with flight length.



9.3 Route Performance

Presents the busiest routes as a sorted bar chart, a detailed reference table combining flight count, average duration, and overnight count per route, and a source/destination traffic comparison to identify the busiest originating and arrival airports.



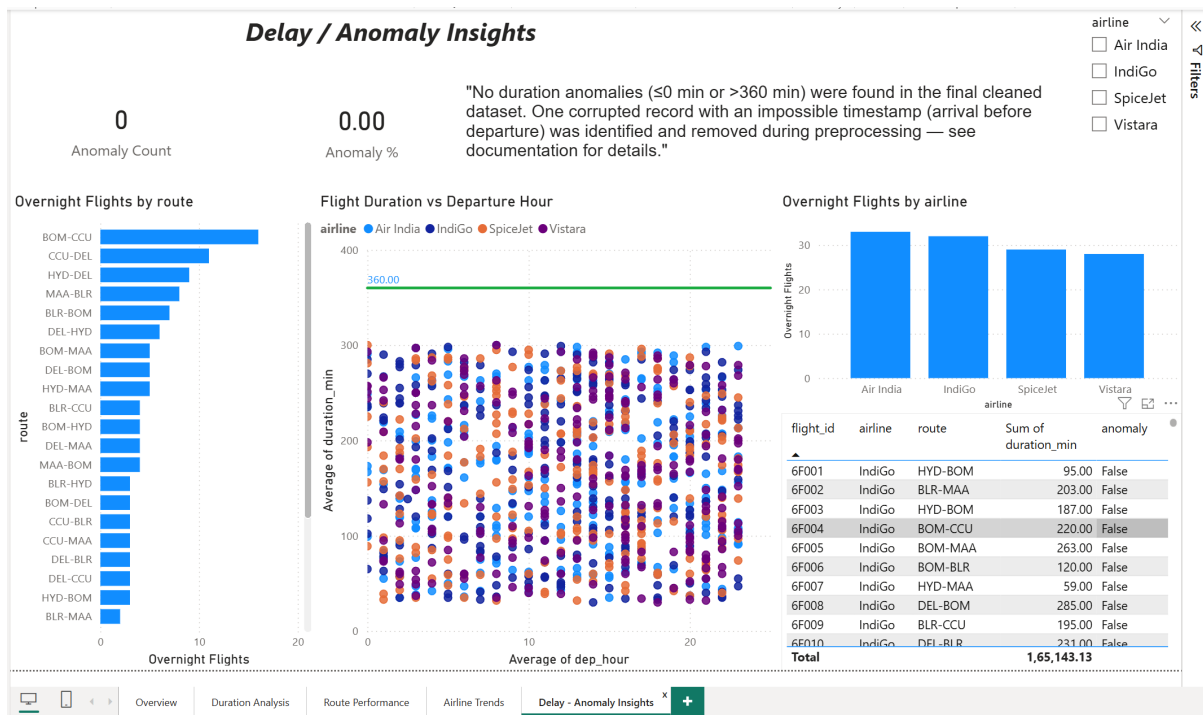
9.4 Airline Trends

Shows the overall market share of each airline, a combined view of average duration against anomaly count per airline, a day-by-day flight volume trend line per airline, and an overnight-flight breakdown by airline.



9.5 Delay / Anomaly Insights

Reports the zero-anomaly finding explicitly (with accompanying explanatory text), visualizes the full duration distribution against the anomaly threshold line (360 minutes) to demonstrate that no flight approaches the threshold, and provides supplementary overnight-flight breakdowns by route and airline as a related operational-irregularity view.



10 Conclusion

The pipeline successfully ingests, validates, cleans, and transforms the raw ASG Airlines flight dataset into a reliable, analysis-ready dataset. The key technical challenge — correctly identifying and handling overnight flights — required revisiting an initial assumption about how the source data represented cross-day timestamps; the corrected approach, based on calendar-date comparison rather than raw timestamp comparison, resolved this correctly and additionally surfaced one genuinely corrupted record that had been mis-attributed to the overnight case. The resulting dataset and dashboard provide ASG Airlines with reliable visibility into flight duration, route traffic, airline performance, and operational anomalies.